HDOC-66

Recursive Multiplication of Two Square Matrices

1. Problem Statement

Write a recursive C program to multiply two square matrices. Iterative loops must not be used for the matrix multiplication logic.

2. Algorithm / Logic

1. Input the order n of the two square matrices.

2. Read the elements of both matrices recursively using readMatrix().

3. For each result element C[row][col], recursively calculate the dot product of row row of matrix A and column col of matrix B.

4. The function multiplyCell() uses the recurrence: A[row][k] * B[k][col] + multiplyCell(..., k + 1, n).

5. When k becomes n, return 0 to terminate the dot-product recursion.

6. Use multiplyMatrices() recursively to move from one result cell to the next. When a column ends, continue with the first column of the next row.

7. Print the product matrix recursively using printMatrix().

3. Function Used

Function	Purpose
readMatrix()	Reads matrix elements recursively.
multiplyCell()	Recursively computes one element of the product matrix.
multiplyMatrices()	Recursively computes all cells of the product matrix.
printMatrix()	Displays the result matrix recursively.
main()	Accepts input, validates the order, and calls the recursive functions.

4. Header File

#include <stdio.h> - provides standard input/output functions such as printf() and scanf().

5. C Program

```
#include <stdio.h>
#define MAX 10
```

```
void readMatrix(int a[MAX][MAX], int n, int i, int j) {
   if (i == n) return;
   if (j == n) {
      readMatrix(a, n, i + 1, 0);
      return;
   }
   scanf("%d", &a[i][j]);
   readMatrix(a, n, i, j + 1);
}

int multiplyCell(int a[MAX][MAX], int b[MAX][MAX], int row, int col, int k, int n) {
   if (k == n) return 0;
   return a[row][k] * b[k][col] + multiplyCell(a, b, row, col, k + 1, n);
}

void multiplyMatrices(int a[MAX][MAX], int b[MAX][MAX], int c[MAX][MAX], int n, int i, int j) {
   if (i == n) return;
   if (j == n) {
      multiplyMatrices(a, b, c, n, i + 1, 0);
      return;
   }
   c[i][j] = multiplyCell(a, b, i, j, 0, n);
   multiplyMatrices(a, b, c, n, i, j + 1);
}

void printMatrix(int c[MAX][MAX], int n, int i, int j) {
   if (i == n) return;
   if (j == n) {
      printf("\n");
      printMatrix(c, n, i + 1, 0);
      return;
   }
   printf("%d\t", c[i][j]);
   printMatrix(c, n, i, j + 1);
}

int main(void) {
   int a[MAX][MAX], b[MAX][MAX], c[MAX][MAX], n;
   printf("Enter order of square matrices: ");
   scanf("%d", &n);
   if (n <= 0 || n > MAX) {
      printf("Invalid matrix order.\n");
      return 0;
   }
   printf("Enter elements of first matrix:\n");
   readMatrix(a, n, 0, 0);
   printf("Enter elements of second matrix:\n");
   readMatrix(b, n, 0, 0);
   multiplyMatrices(a, b, c, n, 0, 0);
   printf("Product matrix:\n");
   printMatrix(c, n, 0, 0);
   return 0;
}
```

6. Explanation

For ordinary matrix multiplication, each element of the result matrix is the sum of products of corresponding elements from one row of the first matrix and one column of the second matrix. In this program, that summation is performed by multiplyCell() recursively instead of by a loop. The function multiplyMatrices() recursively advances through every row and column of the result matrix. Therefore, the multiplication logic contains no iterative loop. The input and output traversal are also implemented recursively.

For an n x n matrix, n^2 result elements are produced and each requires n multiplications, so the time complexity is O(n^3). The recursive call stack requires additional space; the deepest combined recursion is O(n^2) for matrix traversal.

7. Test Cases

Test	Matrix A	Matrix B	Expected Product
1	[[1, 2], [3, 4]]	[[5, 6], [7, 8]]	[[19, 22], [43, 50]]
2	[[2]]	[[3]]	[[6]]
3	Identity 2x2	[[4, 5], [6, 7]]	[[4, 5], [6, 7]]
4	[[1, 0], [0, 1]]	[[0, 0], [0, 0]]	[[0, 0], [0, 0]]

8. Sample Output

```
Enter order of square matrices: 2
Enter elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8
Product matrix:
19  22
43  50
```

9. Result

The program successfully multiplies two square matrices using recursion. The multiplication and traversal logic are performed without iterative loops, and the correct product matrix is obtained.